

National University of Computing and Emerging Sciences				
	Course:	Data Structures	Course Code:	CS 2001
	Program:	BS (CS)	Semester:	Fall 2025
	Name:		Section:	BCS-3D
	Roll No.		Quiz-1 (B)	

Question 1 [4 marks]

Write the function **void DeleteHead(Node* head)** that deletes the head of a linked list

<pre>//Consider the following node structure struct Node { int data; Node* next; };</pre>	<pre>void deleteHead(Node*& head) { if (head == nullptr) return; Node* temp = head; head = head->next; delete temp; }</pre>
--	---

Question 2 [3 marks]

You are given the **Node** structure (given above) and the function **void tricky(Node* head)**. Dry run the following linked list

47 → 64 → 98 → 34 → 20 → 90 → NULL

Write down the output produced by the function

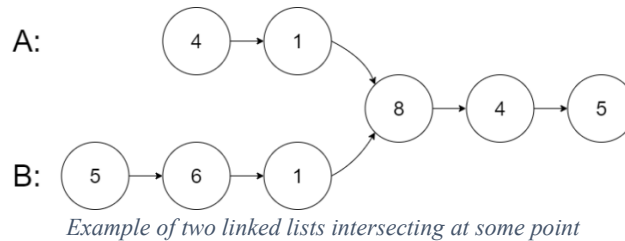
<pre>void tricky(Node* head) { Node* p = head; Node* q = head->next; while (q != NULL && q->next != NULL) { cout << p->data << " "; p = q->next; q = p->next; } }</pre>	Output: 47, 98
--	-------------------------------------

Question 3 [8 marks]

Two singly linked lists may **intersect** at some point (i.e., they share some common nodes by reference, not just by value).

Write a function `Node* getIntersectionNode(Node* headA, Node* headB)` to **find the intersection point of two linked lists**:

- If the two linked lists intersect, return the pointer to the first common node.
- If they do not intersect, return NULL.
- You must solve this **without modifying the linked lists** and without using extra data structures like arrays or sets.



Solution:

```
Node* getIntersectionNode(Node* headA, Node* headB) {  
    if (headA == NULL || headB == NULL) return NULL;  
  
    Node* a = headA;  
    Node* b = headB;  
  
    while (a != b) {  
        if (a != NULL)  
            a = a->next;  
        else  
            a = headB;  
  
        if (b != NULL)  
            b = b->next;  
        else  
            b = headA;  
    }  
    return a; // intersection node or NULL  
}
```